

Video Processing – Final Project

Report

Course 0512.4263 2025/2026

Overview

The project processes a shaky video of a walking person in four stages. Each stage consumes the previous stage's output:

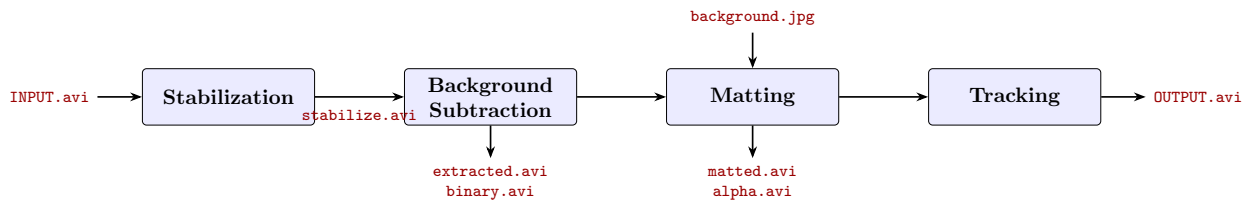


Figure 1: The processing pipeline. Blue blocks are the implemented stages; red names are the video/image artifacts each stage reads or writes.

1 Stabilization

Input. INPUT.avi – a shaky, handheld color video of a person walking.

Expected output. stabilize_ID1_ID2.avi – the same video, in color, with the camera motion cancelled: every frame is warped onto the coordinate system of the first frame, so the background stays fixed for the next stage. Black regions near the frame borders are a known side effect of the warp and are allowed.

Algorithm. We detect Shi–Tomasi corners and track them with pyramidal Lucas–Kanade optical flow, both exactly as taught in the course (and implemented in Exercise 2; here we call OpenCV's implementation for speed). A robust fit with RANSAC, also as seen in the lectures, maps the tracked corners' positions onto a single global motion model per frame, and rejects the corners that sit on the walking person, whose motion disagrees with the camera's. The frame is then warped once, in color, by the fitted transform.

Two choices differ from the straightforward recipe and need a word:

- *Anchored fit instead of composed transforms.* The usual approach estimates the motion from frame t to $t - 1$ and composes all these transforms to reach frame 0. Composing ~ 200 noisy estimates accumulates error, and the background slowly drifts. Instead, every tracked corner

carries its frame-0 position (its “anchor”), and each frame we fit the transform *directly* from current positions to anchors. With no composition chain, drift cannot build up. Corners lost to occlusion or RANSAC rejection are replaced by freshly detected ones, anchored through the current fit.

- *Homography as the motion model.* From the transformation ladder seen in class (translation $\rightarrow$ affine $\rightarrow$ homography) we use the homography. A handheld camera also rotates slightly out of the image plane, which creates a small perspective (keystone) distortion between frames; similarity and affine models cannot represent it, and with them a residual wobble remains visible.

Result. In the output video the scene is static: walls, doors and posters stay in place through the entire clip and only the person moves. Averaging all output frames gives a sharp image of the corridor with a faint ghost of the walking person, which is exactly the behaviour a stabilized static background should show. Narrow black wedges appear at the frame borders, as expected.

2 Background Subtraction

Input. `stabilize_ID1_ID2.avi` – the stabilized color video from stage 1.

Expected output. Two videos: `extracted_ID1_ID2.avi`, where the person keeps the original pixel values and everything else is black, and `binary_ID1_ID2.avi`, the matching mask (person = 1, saved scaled to 255).

Algorithm. The core is the per-pixel Gaussian Mixture Model background model from the lectures, used as taught: each pixel holds $K = 4$ Gaussians $(\mu_k, \sigma_k, \omega_k)$, a new sample matches a Gaussian within 2.5σ , matched Gaussians are updated and their weight grows, unmatched ones decay, the weakest is replaced on no match, and the background Gaussians are the top ones by ω_k/σ_k up to a cumulative weight $T = 0.7$, inside the lecture’s own suggested range of 0.7–0.9. The departures from the lecture version:

- *Offline instead of online.* The lecture’s algorithm serves a live camera. We have the full video on disk, so the model is trained on the whole clip first and only then is every frame classified, against the final model.
- *Median/MAD initialization.* The main Gaussian of every pixel starts at the pixel’s temporal median color, with σ from the median absolute deviation. The person covers any single pixel for a minority of the frames, so the median already *is* the background; starting there instead of from a blind guess avoids pixels that never converge within the clip’s length.
- *CIE-Lab color space.* Matching runs in Lab with the illumination channel down-weighted: $d^2 = 0.25 \Delta L^2 + \Delta a^2 + \Delta b^2$. Shadows and lighting changes live almost entirely in L , while the person differs in chroma, so this makes the classifier insensitive to illumination but sharp on real color change. A pixel whose L drops to 50–96% of the background’s while a, b stay within 8 units is labelled a cast shadow and kept as background.
- *Cleanup.* Morphological opening and closing (recommended in the lecture notes), keeping every connected component at least 20% of the largest one (dark clothing can split a limb off the torso; keeping only the largest deletes real body parts), hole filling, and a smooth upscale of the mask back to full resolution.

Result. The binary mask holds the complete silhouette – head, arms, legs, shoes – in all frames, with no background patches, and the person’s cast shadow on the floor is not included. The extracted video shows the person cleanly cut out on black, with a smooth boundary, ready for matting.

3 Matting

Input. Three artifacts: `stabilize_ID1_ID2.avi` (the stabilized color video), `binary_ID1_ID2.avi` (the person mask from stage 2), and `background.jpg` (the new background image, resized to the video resolution).

Expected output. Two videos: `matted_ID1_ID2.avi`, the person composited onto the new background, and `alpha_ID1_ID2.avi`, the per-pixel opacity $\alpha \in [0, 1]$ stored scaled to $[0, 255]$ and replicated to three channels.

Algorithm. We implement the video-matting pipeline of Lecture 10, applied independently to every frame: *trimap* $\rightarrow$ *color model (KDE)* $\rightarrow$ *opacity map from geodesic distances* $\rightarrow$ *compositing with the pure foreground color*. To keep the runtime low, all work is confined to a region of interest (ROI) around the person’s bounding box (`utils.bbox_of_mask` plus a margin); everything outside is trivially background and is copied straight from the new background image. The four steps are:

1. *Trimap*. Eroding and dilating the binary mask with a disc of radius $\rho = 7$ (RHO) splits the ROI into three regions: the eroded core is confident foreground ($\alpha = 1$), everything outside the dilation is confident background ($\alpha = 0$), and the ring between them is the undecided band φ of width $\approx 2\rho$ where α must be solved.
2. *Color model*. Foreground and background color densities $f(c | F)$, $f(c | B)$ are estimated in CIE-Lab from pixels adjacent to the band (where the statistics are local and reliable) and combined by Bayes with uniform priors into the posterior $P(F | c) = \frac{f(c | F)}{f(c | F) + f(c | B)}$.
3. *Opacity*. We build an 8-neighbour pixel graph over the band and its confident rims whose edge cost is the posterior jump $|P(F | c(u)) - P(F | c(v))| + \varepsilon$, and run multi-source Dijkstra from each rim to obtain the geodesic distances D_F , D_B . The opacity blends distance and color:

$$w_F = D_F^{-r} P(F | c), \quad w_B = D_B^{-r} P(B | c), \quad \alpha = \frac{w_F}{w_F + w_B}, \quad r = 2.$$

4. *Compositing*. The naive blend $\alpha c + (1 - \alpha)B_{\text{new}}$ would drag the old background’s color into the new one. Instead, for every band pixel we take the $N_{\text{cand}} = 7$ nearest confident foreground and background pixels and pick the pair (x_F^*, x_B^*) whose blend best reconstructs the observed color, $\arg \min_{x_F, x_B} \|\alpha c(x_F) + (1 - \alpha)c(x_B) - c(x)\|^2$, then composite with the recovered pure foreground color, $J = \alpha c(x_F^*) + (1 - \alpha)B_{\text{new}}$.

Points that differ from the lecture recipe or were not shown verbatim in class:

- *Trimap from a dense mask (modification)*. The lecture builds the trimap from a few user scribbles and classifies the rest with KDE and geodesics. Here stage 2 already provides a dense foreground/background labelling, so the trimap is obtained geometrically by erode/dilate – no scribbles and no user input.
- *Fast binned KDE (not taught verbatim)*. A direct Parzen estimate sums a Gaussian over all samples for every query pixel, which is $O(|\Omega| \cdot |\text{samples}|)$. We instead quantize the Lab colors into a 64^3 histogram of counts and smooth it once with a separable 3-D Gaussian (three 1-D convolutions, `scipy.ndimage.gaussian_filter`); evaluating the density at any pixel is then a single table lookup. This binned low-rank scheme is what makes the KDE affordable per frame (`kde_3d` below).
- *Geodesic distance by graph + Dijkstra (not taught verbatim)*. The lecture defines the geodesic length as a line integral of the posterior gradient. We discretize it exactly: the graph’s edge weight is the posterior difference between neighbours, and the minimum

path cost to a confident region – D_F, D_B – is computed with multi-source Dijkstra (`scipy.sparse.csgraph.dijkstra`).

- *KD-tree candidate window (not taught verbatim)*. The nearest confident pixels for the compositing search are found with a KD-tree (`scipy.spatial.cKDTree`) instead of a brute-force scan.

None of these library calls performs the matting itself: Dijkstra, the KD-tree and the Gaussian filter are generic numerical helpers, and every decision (trimap, posterior, opacity, compositing) is written explicitly. No deep-learning model is used, and we deliberately avoid the OpenCV-contrib matting routines (the “way out” that caps the project at 70 points). The four steps above are chained per frame by `process_frame`, with `kde_3d` and `pure_composite` implementing the fast KDE and the pure-color compositing.

Result. The alpha video is white on the body and black on the background, with a thin grey ramp exactly along the silhouette; the matted video shows the person on the new background with clean edges and no colored halo from the old corridor, confirming that the pure-color compositing step works. Because all work is confined to the ROI and the KDE is binned, the stage stays well within the runtime budget.

4 Tracking

Input. The composited video `matted_ID1_ID2.avi` from stage 3. The mask video `binary_ID1_ID2.avi` is read only, to initialize the tracker automatically.

Expected output. `OUTPUT_ID1_ID2.avi`, the matted video with a green rectangle drawn around the tracked person, and `tracking.json`, mapping each frame “1”...“N” to the box [ROW, COL, HEIGHT, WIDTH] (top-left corner).

Algorithm. We track with the particle filter (Condensation) from the tracking lecture, implemented in Exercise 3. Each particle is a state $s = [x_c, y_c, w_{1/2}, h_{1/2}, v_x, v_y]$ (center, half-size, velocity), and $N = 100$ particles are propagated each frame through the four Condensation steps:

1. *Sample*. Draw N particles from the previous set in proportion to their weights, by inverting the CDF C : a uniform $r \in [0, 1]$ selects the first index j with $C[j] \geq r$ (`sample_particles`).
2. *Predict*. Apply the constant-velocity model $x \leftarrow x + v_x$, $y \leftarrow y + v_y$ and add white Gaussian noise to position and velocity (`predict_particles`).
3. *Measure*. Describe each particle’s rectangle by a normalized $16 \times 16 \times 16$ color histogram (4-bit-per-channel quantization), score it against the reference histogram q with the Bhattacharyya coefficient $BC(p, q) = \sum \sqrt{p_i q_i}$, and set the weight $w = \exp(20 \cdot BC)$.
4. *Estimate*. Normalize the weights, rebuild the CDF, and output the weighted-mean state $E[s] = \sum_i W_i s_i$ as the tracked box.

Modifications to the Exercise-3 version:

- *Automatic initialization*. Exercise 3 hard-codes the first box. We instead take the first non-empty binary mask (`find_initial_state`): its bounding box gives s_{initial} , so the tracker bootstraps from the background-subtraction output with no manual input. Frames before that index simply report the initial box.
- *Masked reference histogram*. The standard color-histogram tracker builds q from the whole initial rectangle. We build it from the person pixels only, using the binary mask (`compute_reference_histogram`); a candidate box that merely covers background then cannot score a high Bhattacharyya coefficient, which sharpens the tracker’s lock on the person.

- *Noise magnitudes.* Exercise 3 used small images and small noise; here the video is full-HD and the person walks ~ 10 px/frame and turns around mid-clip, so the diffusion is scaled up to $\sigma_{\text{pos}} = 10$, $\sigma_{\text{vel}} = 3$ px. The exercise leaves these magnitudes as a free choice.

The Bhattacharyya coefficient (the overlap of two color distributions) and the CDF-inversion re-sampling are both from the exercise/lecture. No library routine performs the tracking – every step is hand-written in NumPy, and only `cv2.rectangle` is used to draw the box – and no deep-learning or built-in tracker is involved, so the 70-point contrib cap does not apply. With $N = 100$ particles the stage runs comfortably inside the runtime budget.

Result. The green box follows the person through the whole clip, staying locked on even when they turn around, and `tracking.json` contains one [ROW, COL, HEIGHT, WIDTH] entry per frame, keyed from “1”.